**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM**
**KHOA KỸ THUẬT MÁY TÍNH**

**KIẾN TRÚC MÁY TÍNH**

# LAB 01

## LÀM QUEN MARS: XỬ LÝ I/O và CẤU TRÚC ĐIỀU KIỆN

| | | | |
|---|---|---|---|
| **Họ và tên** | Huỳnh Phạm Ngọc Thắng | **MSSV** | 25521672 |
| **Lớp** | CE119.R11.VNTN.1 | **Ngày thực hành** | 6/10/2026 |
| **Tự chấm** | 9 | **Link git** | https://github.com/hpnt1912/COMPUTER-ARCHITECTURE/tree/main/mars_mips/lab1 |

## 1. Tổng quan buổi thực hành

Sinh viên dùng trình mô phỏng **MARS 4.5** để biến kiến thức Chương 2 (kiến trúc bộ lệnh MIPS: toán hạng, số có dấu/không dấu, biểu diễn lệnh, lệnh điều kiện và nhảy) thành các thí nghiệm dự đoán – lập trình – quan sát – giải thích. Tình huống xuyên suốt: một chương trình tính tiền điện sinh hoạt theo biểu giá bậc thang, kèm một đoạn giai điệu ngắn phát bằng syscall MIDI để báo hoàn tất.

Sau buổi thực hành, Sinh viên có thể:

- Soạn, Assemble, chạy từng bước và gỡ lỗi chương trình MIPS32 trên MARS; dùng syscall nhập/xuất, thời gian hệ thống và MIDI.
- Phân biệt dòng lệnh nguồn, giả lệnh và lệnh cơ bản; đọc cột Code/Basic của Text Segment và tự giải mã các trường của lệnh dạng R, dạng I và độ lệch của lệnh nhánh.
- Hiện thực cấu trúc điều kiện và vòng lặp bằng `slt`/`sltu`/`beq`/`bne`; truy cập mảng trong bộ nhớ bằng địa chỉ cơ sở và chỉ số.
- Phân biệt so sánh có dấu và không dấu khi kiểm tra miền giá trị của dữ liệu nhập.
- Dùng Instruction Counter để đo số lệnh thực thi, xây dựng mô hình chi phí theo dữ liệu vào và đánh giá một thay đổi thiết kế bằng số liệu đo.

| Phần | Nội dung | Thời gian gợi ý | Điểm |
|---|---|---|---|
| **Phần 0** | Làm quen MARS, kiểm tra môi trường | 15 phút | |
| **TN 1** | Từ lệnh nguồn đến mã máy: giả lệnh, Text Segment, Instruction Counter | 20 phút | 1,5 |
| **TN 2** | Tính tiền điện bậc thang: nhập có kiểm tra, biểu giá lưu trong bộ nhớ | 70 phút | 4,0 |
| **TN 3** | Phát giai điệu bằng syscall MIDI và đo thời gian chạy | 25 phút | 2,0 |
| **TN 4** | Đo số lệnh và giải mã lệnh trên chương trình của mình | 30 phút | 2,0 |
| **Kết** | Tổng kết, phản tư, nộp bài | 20 phút | 0,5 |
| **Tổng** | | **180 phút** | **10,0/10,0** |

### Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- **Tham số cá nhân.** `a` là chữ số cuối của MSSV, `b` là chữ số liền trước. Mọi số liệu riêng của bài (đơn giá, nhịp độ, nhạc cụ, giá trị kiểm thử) được tính từ `a` và `b`; điền vào Bảng 1 trước khi bắt đầu.
- **Quy ước mã nguồn.** Mỗi chương trình có `.data`, `.text`, nhãn `main` và kết thúc bằng syscall 10. Chuỗi in ra và chú thích dùng ASCII không dấu. Ghi chú vai trò thanh ghi theo quy ước: `$t*` giá trị tạm, `$s*` giá trị cần giữ qua syscall, `$v0` số hiệu syscall, `$a0`–`$a3` đối số.
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi chạy trên MARS; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.

## PHẦN 0 — Làm quen MARS (15 phút)

Giảng viên hướng dẫn nhanh giao diện MARS. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng dưới đây (bỏ qua nếu đã thành thạo).

| ☑ | Kỹ năng | Gợi ý |
|---|---|---|
| ☑ | Tạo file mới, lưu với tên `Lab01_<MSSV>_dien.asm` | Mỗi chương trình nộp là một file riêng; xem quy ước ở cuối tài liệu. |
| ☑ | Assemble (Run ▸ Assemble, F3), chạy (Run ▸ Go, F5), chạy từng lệnh (F7), lùi một lệnh (F8), đặt lại (F12) | Sau khi sửa mã phải Assemble lại. Lỗi Assemble hiện ở tab Mars Messages. |
| ☑ | Kiểm tra Settings: bỏ chọn **Delayed branching**; chọn **Permit extended (pseudo) instructions and formats** | Hai thiết lập này ảnh hưởng trực tiếp đến mã máy và số lệnh đếm được. |
| ☑ | Đọc các vùng của tab Execute: Text Segment (Address, Code, Basic, Source), Data Segment, Registers | Tích **Labels** ở dưới Data Segment để xem địa chỉ của nhãn dữ liệu. |
| ☑ | Đặt breakpoint (ô Bkpt) và chạy từng bước để quan sát thanh ghi đổi màu | Thanh ghi vừa bị ghi sẽ được tô sáng. |
| ☑ | Mở Tools ▸ Instruction Counter và bấm **Connect to MIPS** trước khi chạy | Công cụ chỉ đếm khi đã kết nối; bấm **Reset** để đếm lại từ 0. |
| ☑ | Tra bảng syscall trong Help ▸ Help (thẻ MIPS, mục Syscalls) | Bảng tóm tắt các syscall dùng trong buổi này nằm ngay dưới. |
| ☑ | Chụp màn hình cửa sổ MARS và dán vào khung ảnh của file này | Chụp cả vùng Run I/O lẫn vùng mã đang xét. |

### Các syscall và lệnh dùng trong buổi này

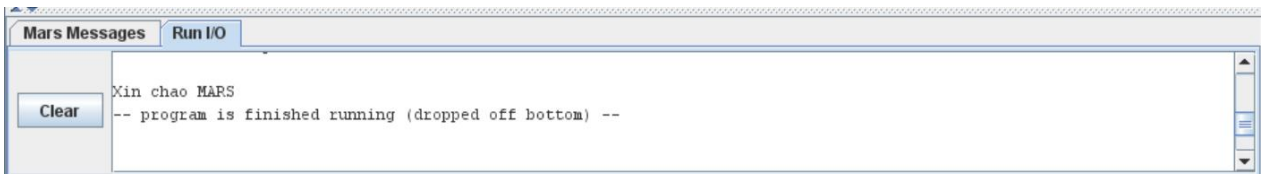
Đặt số hiệu dịch vụ vào `$v0`, đối số vào `$a0`–`$a3`, rồi thực hiện `syscall`.

| $v0 | Dịch vụ | Đối số | Kết quả |
|---|---|---|---|
| 1 | In số nguyên | `$a0` = giá trị | |
| 4 | In chuỗi kết thúc bằng 0 | `$a0` = địa chỉ chuỗi | |
| 5 | Đọc số nguyên có dấu | | `$v0` = giá trị đọc được |
| 10 | Kết thúc chương trình | | |
| 30 | Lấy thời gian hệ thống (ms) | | `$a0` = 32 bit thấp, `$a1` = 32 bit cao |
| 31 | Phát một nốt MIDI, trả về ngay | `$a0` cao độ (0–127), `$a1` độ dài (ms), `$a2` nhạc cụ (0–127), `$a3` âm lượng (0–127) | |
| 32 | Chờ (ngủ) | `$a0` = số ms | |
| 33 | Phát một nốt MIDI, chờ đến khi nốt kết thúc | như syscall 31 | |

Lệnh `mul rd, rs, rt` nhân hai số và ghi 32 bit thấp của tích vào `rd` (cơ chế bên trong sẽ học ở Chương 3, ở đây dùng như một lệnh có sẵn).

| Mục | Ghi nhận |
|---|---|
| Phiên bản MARS đang dùng (Help ▸ About) | |
| Máy có phát được âm thanh qua MIDI không? (Có / Không) | |

**Bước 0 – Kiểm tra nhanh.** Tạo chương trình in dòng chữ `Xin chao MARS` rồi kết thúc (chỉ dùng syscall 4 và 10). Assemble, chạy và dán ảnh chụp vùng Run I/O vào khung dưới. File này chỉ để làm quen, không nộp.




**Tham số cá nhân**

Điền `a`, `b` rồi tính các giá trị còn lại. Các giá trị này dùng cho TN 2 và TN 3.

**Bảng 1.** Tham số cá nhân

| Đại lượng | Cách tính | Giá trị của Sinh viên |
|---|---|---|
| a | Chữ số cuối của MSSV | 2 |
| b | Chữ số liền trước chữ số cuối của MSSV | 7 |
| Đơn giá bậc 1 (đồng/kWh) | 1600 + 20*a | 1640 |
| Số nửa cung dịch giọng | a | 2 |
| Độ dài một đơn vị nhịp (ms) | 150 + 10*b | 220 |
| Nhạc cụ (số hiệu chương trình MIDI) | Phần dư của b khi chia 4 là 0, 1, 2, 3 tương ứng 0, 24, 40, 73 | 73 |
| Số kWh kiểm thử cá nhân x_b | 100 + 25*b | 275 |

# THÍ NGHIỆM 1 — Từ lệnh nguồn đến mã máy

Thời lượng gợi ý: 20 phút · File: tn1.asm (không nộp) · Điểm: 1,5

### Tình huống

Trước khi viết chương trình lớn, kỹ sư cần biết một dòng mã nguồn thực sự trở thành bao nhiêu lệnh máy: số lệnh này quyết định kích thước chương trình và (cùng với số lệnh thực thi) thời gian chạy. Thí nghiệm này khảo sát một chương trình nhỏ để làm rõ ba con số: số dòng nguồn, số lệnh cơ bản trong bộ nhớ và số lệnh thực sự được thực thi.

**Chương trình khảo sát tn1.asm (gõ lại nguyên văn, chưa Assemble)**

```
        .data
msg:    .asciiz "Ket qua: "
        .text
main:   li   $t0, 25
        li   $t1, 70000
        add  $t2, $t0, $t1
        blt  $t0, $t1, small
        li   $t2, 0
small:  la   $a0, msg
        li   $v0, 4
        syscall
        move $a0, $t2
        li   $v0, 1
        syscall
        li   $v0, 10
        syscall
```

**Bước 1.1 – Dự đoán trước khi Assemble.** Chưa chạy MARS. Đọc chương trình và điền cột Dự đoán của Bảng 2 và Bảng 3.

**DỰ ĐOÁN** Mỗi dòng nguồn ở Bảng 2 sinh ra bao nhiêu lệnh cơ bản? Hai lệnh trong Bảng 3 là bao nhiêu dòng nguồn, lệnh cơ bản (trong bộ nhớ) và lệnh được thực thi? Chương trình in ra gì?

**Bước 1.2 – Assemble và quan sát.** Assemble, đọc Text Segment, điền cột Quan sát. Mở Instruction Counter (Connect to MIPS), chạy chương trình và đọc tổng số lệnh.

**QUAN SÁT** Cột Basic cho biết các lệnh cơ bản thực sự nằm trong bộ nhớ; cột Source là dòng nguồn tương ứng.

**Bảng 2.** Số lệnh cơ bản sinh ra từ mỗi dòng nguồn

| Dòng nguồn | Dự đoán (số lệnh) | Quan sát (số lệnh) | Các lệnh cơ bản quan sát được (cột Basic) |
|---|---|---|---|
| `li $t0, 25` | 1 | 1 | addiu $8, $0, 0x00000019 |
| `li $t1, 70000` | 1 | 2 | lui $1, 0x00000001<br>ori $9, $1, 0x00001170 |
| `blt $t0, $t1, small` | 2 | 2 | slt $1, $8, $9<br>bne $1, $0, 0x00000001 |
| `la $a0, msg` | 1 | 2 | lui $1, 0x00001001 |
| `move $a0, $t2` | 1 | 1 | addu $4, $0, $10 |

**Bảng 3.** Số dòng lệnh nguồn, lệnh cơ bản, lệnh được thực thi của chương trình khảo sát

| Đại lượng | Dự đoán | Quan sát | Nguồn đọc kết quả |
|---|---|---|---|
| Số dòng lệnh nguồn trong `.text` | 13 | 13 | Đếm trong Source |
| Số lệnh cơ bản trong bộ nhớ (tĩnh) | 1 | 16 | Số hàng của Text Segment |
| Số lệnh được thực thi (động) | 12 | 15 | Instruction Counter, mục Total |
| Chuỗi in ra Run I/O | 1 | Ket qua: 70025 | Cửa sổ Run I/O |

**Câu 1.1 (Hiểu).** Vì sao `li $t0, 25` chỉ cần một lệnh cơ bản còn `li $t1, 70000` cần hai? Giá trị 70000 không vừa trường nào của lệnh dạng I? Thanh ghi `$at` (số 1) xuất hiện ở đâu trong Text Segment và đóng vai trò gì?

*Trả lời:*
*- lệnh li là thuộc dạng R[rd] = immediate 16 bit. Nên số tối đa có thể nạp vào thanh ghi là 2^16 - 1: 25 < 2^16 - 1 = 65535 < 70000*
*- giá trị 70000 không vừa trường immediate của lệnh dạng I*
*- $at xuất hiện ở address 0x00400004 lệnh lui $1, 0x00000001, đóng vai trò là lấy 16 bit cao nhất để load vào thanh ghi $1. lúc này $1 sẽ là: 0x00010000 sau đó ori $9, $1, 0x00001170 sẽ đúng ra 0x0001170 là số 70000*

**Câu 1.2 (Phân tích).** Giải thích vì sao ba con số ở Bảng 3 khác nhau. Trong hai lệnh `blt` và `li $t2, 0`, lệnh nào góp phần làm số lệnh thực thi nhỏ hơn số lệnh trong bộ nhớ và vì sao?

*Trả lời:*
*- Hàng 1 là tính các dòng lệnh sau .text trở đi, hàng 2 là số lệnh trong bộ nhớ thực thi mã giả của mình, hàng 3 là số lệnh được thực thi sau khi sau khi đã thực hiện các câu lệnh rẽ nhánh / điều kiện.*
*- blt giúp làm số lệnh thực thi nhỏ hơn số lệnh trong bộ nhớ vì nó là câu lệnh rẽ nhánh. Nếu điều kiện đúng thì nó sẽ nhảy tới câu lệnh phía sau. Ví dụ:* `blt $t0, $t1, small` *nếu $t0 nhỏ hơn $t1 thì sẽ nhảy sang lệnh small*

**Bước 1.3 – Giải mã bằng tay.** Đọc mã hex của lệnh `add $t2, $t0, $t1` ở cột Code **trước**, tách các trường rồi so với cột Basic. Làm tương tự với lệnh nhánh do `blt` sinh ra.

**Bảng 4.** Giải mã lệnh dạng R: `add $t2, $t0, $t1`

| Địa chỉ | Mã hex (Code) | opcode | rs | rt | rd | shamt | funct |
|---|---|---|---|---|---|---|---|
| 0x0040000c | 0x01095020 | 000000 | 01000 | 01001 | 01010 | 00000 | 100000 |

**Bảng 5.** Giải mã lệnh nhánh sinh ra từ `blt` (lệnh thứ hai của cặp)

| Địa chỉ | Mã hex (Code) | opcode | rs | rt | immediate (thập phân, có dấu) | Địa chỉ nhãn `small` |
|---|---|---|---|---|---|---|
| 0x00400014 | 0x14200001 | 000101 | 00001 | 00000 | 0000000000000001 | 0x0040001c |

**Câu 1.3 (Áp dụng).** Viết phép tính cho thấy trường immediate của lệnh nhánh ở Bảng 5 được suy ra từ địa chỉ của nó và địa chỉ nhãn `small` (nêu rõ vì sao có số hạng 4 và phép chia 4). Nếu chèn thêm 3 lệnh cơ bản vào giữa `blt` và `small`, trường immediate đổi thành bao nhiêu?

*Trả lời:*
*- Ta có công thức địa chỉ: PC = PC + 4 + BranchAddr mà* (4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 } *tức dịch phải 2 bit là sẽ có PC nhưng vì mình tìm immediate nên mình phải suy ra BranchAddr = PC - (PC + 4) và mình phải dịch trái immidiate 2 bit, tương đương với việc chia 4. --> immediate = (PC - (PC + 4)) / 4*
*- 1 lệnh = 4 bytes --> chèn 3 lệnh = 12 bytes --> 0xC --> địa chỉ small = 0x0040001c + 0x0000000c = 0x00400028*
*Immediate = (0x00400028 - 0x0040018) / 4 = 16 bytes / 4 = 4 --> 0x00000004*

**Bước 1.4 – Dự đoán – quan sát lần hai.** Trong tn1.asm thay `blt` bằng `bgt` (giữ nguyên nhãn và các dòng còn lại).

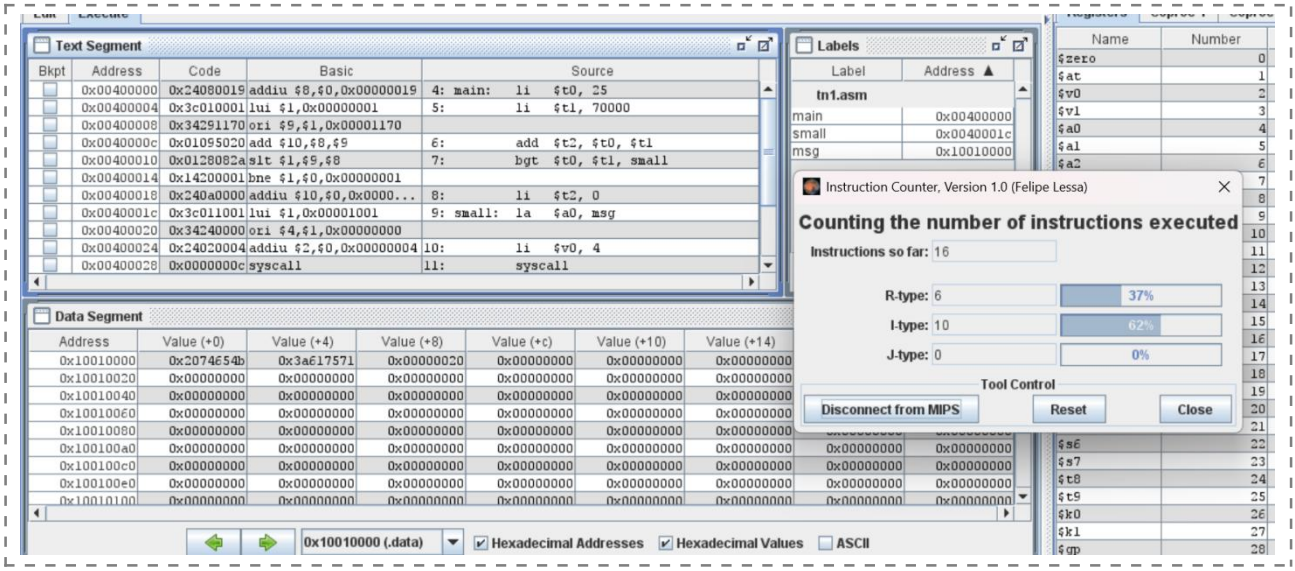
**DỰ ĐOÁN** Số lệnh thực thi bây giờ là bao nhiêu và chuỗi in ra là gì? Ghi dự đoán vào Bảng 6, Assemble, đặt lại Instruction Counter, chạy và ghi quan sát.

**Bảng 6.** Thay `blt` bằng `bgt`

| Đại lượng | Dự đoán | Quan sát |
|---|---|---|
| Số lệnh được thực thi | 16 | 16 |
| Chuỗi in ra | Ket qua: 0 | Ket qua: 0 |

**Câu 1.4 (Phân tích).** Ba lệnh `syscall` trong chương trình có cùng mã máy hay khác nhau (ghi mã hex quan sát được)? Nếu giống nhau thì máy phân biệt dịch vụ cần thực hiện bằng cách nào?

*Trả lời: giống nhau đều là: 0x0000000c*
*Máy phân biệt dịch vụ dựa trên mã số câu lệnh đằng trước ví dụ: li $v0, 4. thì sau khi syscall, cpu sẽ đọc giá trị $v0 thấy số 4 thì sẽ thực hiện dịch vụ in chuỗi string ra màn hình dựa theo bảng dịch vụ syscall*



# THÍ NGHIỆM 2 — Tính tiền điện bậc thang

Thời lượng gợi ý: 70 phút · File nộp: Lab01_<MSSV>_dien.asm · Điểm: 4,0

### Tình huống

Một công ty điện lực cần chương trình tính tiền điện sinh hoạt của một hộ trong một tháng. Điện năng tiêu thụ được chia thành sáu bậc; **mỗi kWh được tính theo đơn giá của bậc chứa nó**, và tổng tiền là tổng của các phần ở từng bậc. Biểu giá dưới đây là biểu giá giả lập dùng cho bài thực hành. Chương trình chạy trên một vi xử lý nhúng nhỏ nên bảng giá phải nằm trong bộ nhớ dữ liệu để thay đổi khi biểu giá đổi mà không phải sửa phần lệnh.

**Bảng 7.** Biểu giá giả lập (đồng/kWh, số nguyên)

| Bậc | Từ (kWh) | Đến (kWh) | Đơn giá (đồng/kWh) |
|---|---|---|---|
| 1 | 0 | 50 | 1600 + 20*a |
| 2 | 51 | 100 | 1660 + 20*a |
| 3 | 101 | 200 | 1950 + 20*a |
| 4 | 201 | 300 | 2500 + 20*a |
| 5 | 301 | 400 | 2850 + 20*a |
| 6 | 401 | 9999 | 3000 + 20*a |

### Đặc tả chương trình

- **Nhập.** In lời nhắc `Nhap so kWh (0..9999): ` rồi đọc một số nguyên có dấu bằng syscall 5.
- **Kiểm tra.** Giá trị hợp lệ khi $0 \le x \le 9999$. Nếu không hợp lệ, in `Gia tri khong hop le, nhap lai.` rồi nhắc nhập lại cho đến khi hợp lệ. Việc kiểm tra miền giá trị chỉ được dùng **một** lệnh so sánh và **một** lệnh nhánh.
- **Dữ liệu.** Ngưỡng các bậc và đơn giá lưu trong `.data` dưới dạng mảng `.word`. Phần tính toán duyệt các mảng này bằng vòng lặp; đổi giá trị trong mảng thì không phải sửa phần lệnh. Không viết riêng mã cho từng bậc.
- **Xuất.** In đúng khuôn `Tien dien: <so tien> dong` rồi xuống dòng, sau đó kết thúc chương trình.
- **Giao diện thanh ghi.** Ghi chú ở đầu file vai trò của từng thanh ghi `$s*` đã dùng.

**Bước 2.1 – Dự đoán bằng tay.** Chưa viết mã. Điền đơn giá 6 bậc của Sinh viên (từ Bảng 1) vào Bảng 8, rồi tính tay tiền điện cho các giá trị x ở Bảng 10 (có thể dùng máy tính bỏ túi).

**DỰ ĐOÁN** Ghi tiền điện dự kiến cho từng x. Chú ý các giá trị nằm đúng trên hoặc sát ranh giới bậc.

**Bảng 8.** Đơn giá 6 bậc theo tham số a của Sinh viên

| Bậc | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| **Đơn giá** | 1640 | 1700 | 1990 | 2540 | 2890 | 3040 |

**Bước 2.2 – Thiết kế trước khi viết mã.** Mô tả thuật toán bằng lời hoặc mã giả (không cần đúng cú pháp) và lập bảng phân công thanh ghi.

**THIẾT KẾ** Ghi rõ điều kiện kết thúc vòng lặp, cách xác định phần điện năng thuộc mỗi bậc và vì sao chương trình không cần biết trước số bậc đã dùng.

*Thuật toán:*
*Khai báo dữ liệu trong .data:*
*Dung lượng từng bậc: 1:50, 2:50, 3:100, 4:100, 5:100, 6:9599*
*DungLuong = [50, 50, 100, 100, 100 ,9599, -1]*
*DonGia = [1640, 1700, 1990, 2540, 2890, 3040]*

*//vòng lặp nhập*
*Int x;*
*Do*
*{*
*Cout << "Nhap so kWh (0..9999)";*
*Cin >> x;*
*If (unsigned(x) >= 10000) cout << "Gia tri khong hop le, nhap lai";*
*Else break;*
*} unsigned(x) >+ 10000;*

*Int res = 0*
*Int i = 0;*
*While ( x > 0 & DungLuong[i] != -1)*
*{*
*limit = DungLuong[i];*
*kWh_bac = (x > limit) ? limit : x;*
*res = res + kWh_bac * DonGia[i];*
*x = x - kWh_bac;*
*i = i + 1;*
*}*

*//in ket qua va ket thuc*
*Cout << "Tien dien: " << res << endl;*

*Điều kiện kết thúc vòng lặp:*
*Khi mà x đã <= 0 tức là đã tính toán xong số kWh và tính toán được tổng số tiền khi sử dụng số kWh đó hoặc DuongLuong[i] = -1 tức là đã tính xong số tiền ở mốc cuối*

*Cách xác định phần điện năng thuộc mỗi bậc:*
*Cho limit là phần DungLuong tối đa của bậc hiện tại và tính toán xem nếu x > limit thì tức là điện năng đang lớn hơn cả DungLuong hiện tại nên chắc chắn số kWh đó phải là tối đa, còn khi x <= limit thì điện năng đó đang thuộc trong khoảng dung lượng của bậc ấy nên kWh sẽ chính là x luôn*

*Chương trình không cần biết trước số bậc đã dùng vì chương trình sử dụng vòng lặp duyệt mảng phần tử cho đến khi điện năng x giảm về 0*

**Bảng 9.** Phân công thanh ghi

| Thanh ghi | Vai trò trong chương trình |
|---|---|
| $s0 | Lưu số kWh tiêu thụ do người dùng nhập(x) |
| $s1 | Lưu tổng số tiền tính được (res) |
| $s2 | Lưu địa chỉ mảng cơ sở DungLuong |
| $s3 | Lưu địa chỉ mảng cơ sở DonGia |
| $t0 | Biến đếm chỉ số vòng lặp (i) |
| $t1 | Lưu dung lượng tối đa của bậc hiện tại (limit) |

| Thanh ghi | Vai trò trong chương trình |
|---|---|
| $t2 | Lưu đơn giá của bậc hiện tại DonGia[i] |
| $t3 | Lưu số kWh của bậc hiện tại (Kwh_bac) |
| $t4 | Thanh ghi tạm tính offset và địa chỉ phần tử mảng |
| $t5 | Thanh ghi tạm để tính toán |
| $v0, a0 | Dùng cho syscall để nhập / xuất dữ liệu |

**Bước 2.3 – Lập trình.** Viết chương trình theo đặc tả trong file `Lab01_<MSSV>_dien.asm`. Assemble, sửa lỗi, chạy từng bước ít nhất một lần để theo dõi vòng lặp qua các bậc.

**LẬP TRÌNH** Khi gỡ lỗi, đặt breakpoint ở đầu thân vòng lặp và theo dõi các thanh ghi tổng tiền, cận dưới và con trỏ vào mảng.

**Bước 2.4 – Kiểm thử và so sánh với dự đoán.** Chạy chương trình lần lượt với các giá trị ở Bảng 10 và Bảng 11.

**QUAN SÁT** Ghi kết quả chương trình in ra và đối chiếu với dự đoán. Nếu lệch, tìm xem dự đoán tay hay chương trình sai, ghi kết luận vào cột cuối.

**Bảng 10.** Tiền điện: dự đoán và quan sát

| x (kWh) | Dự đoán (đồng) | Quan sát (đồng) | Khớp? Nếu lệch, nguyên nhân |
|---|---|---|---|
| 30 | 49200 | 49200 | khớp |
| 50 | 82000 | 82000 | khớp |
| 51 | 83700 | 83700 | khớp |
| 120 | 206800 | 206800 | khớp |
| 250 | 493000 | 49300 | khớp |
| 401 | 912040 | 912040 | khớp |
| x_b = 7 | 11480 | 11480 | khớp |

**Bảng 11.** Dữ liệu nhập không hợp lệ và giá trị biên của miền nhập

| Dữ liệu nhập | Dự đoán hành vi | Quan sát |
|---|---|---|
| 0 | Hợp lệ -> Tien dien: 0 dong | Tien dien: 0 dong |
| 9999 | Hợp lệ -> Tien dien: 30012960 dong | Tien dien: 30089960 dong |
| 10000 | Ko hợp lệ, cho nhập lại | Gia tri khong hop le, nhap lai<br>Nhap so kWh (0..9999): |
| −1 | Ko hợp lệ, cho nhập lại | Gia tri khong hop le, nhap lai<br>Nhap so kWh (0..9999): |

**Câu 2.1 (Phân tích).** Chỉ rõ lệnh so sánh và lệnh nhánh dùng để kiểm tra miền [0, 9999] trong chương trình của Sinh viên. Giải thích vì sao **một** phép so sánh bắt được cả $x < 0$ lẫn $x \geq 10000$.

*Trả lời:*
*- lệnh so sánh: sltiu $t3, $s0, 10000*
*- lệnh nhánh: beq $t3, $zero, NhapLai*
*--> khi x < 0 thì bit 31 của nó là 1, nó sẽ ở dạng số rất lớn khi ở phép so sánh ko dấu (bit số 31 là 1 thì --> 2^31, rất lớn >= 10000), còn khi x >= 10000 ở dạng ko dấu thì bản chất đúng là nó >= 10000.*

**Bước 2.5 – Thí nghiệm có dấu và không dấu.** Đổi lệnh so sánh ở câu 2.1 sang phiên bản so sánh **ngược** kiểu (có dấu thành không dấu hoặc ngược lại), Assemble lại, nhập x = −1.

**DỰ ĐOÁN** Chương trình sẽ làm gì? Ghi dự đoán trước, rồi chạy và ghi quan sát. Sau khi xong, khôi phục lệnh đúng.

| Dự đoán | Quan sát |
|---|---|
| Chương trình chấp nhận x = -1 và sẽ in ra:<br>Tien dien: 0 dong | Tien dien: 0 dong |

**Câu 2.2 (Phân tích).** Giải thích kết quả quan sát bằng cách xét biểu diễn bù hai của −1 và cách hai lệnh so sánh diễn giải cùng một chuỗi bit.

*Trả lời: bù 2 của -1 : 111111... = 0xFFFFFFFF*
*Slti(có dấu): đọc 0xFFFFFFFF là : 2^-31 + 2^30 + 2^29... + 2^0 = -1*
*Sltiu(ko dấu): đọc 0xFFFFFFFF là : 2^31 + 2^30+... là 1 số rất lớn*

*Nên khi dùng stli thì -1 < 10000 đúng nên nó sẽ nhảy tới xu_ly và tại đây x = -1 <= 0 nên lập tức vòng lặp dừng và in ra giá trị res ban đầu khởi tạo là 0 dong*

**Bước 2.6 – Quan sát bộ nhớ.** Mở Tools → tích **Labels**, quan sát cửa sổ Labels và tìm địa chỉ nhãn của mảng đơn giá trong Data Segment. Đọc phần tử thứ 3 (chỉ số 2) ở dạng hex và thập phân.

**QUAN SÁT** Đối chiếu địa chỉ phần tử với địa chỉ nhãn và với lệnh nạp dữ liệu tương ứng trong chương trình.

**Bảng 12.** Mảng đơn giá trong Data Segment

| Địa chỉ nhãn mảng | Địa chỉ phần tử thứ 3 | Giá trị (hex) | Giá trị (thập phân) |
|---|---|---|---|
| 0x1001001c | 0x10010024 | 0x000007c6 | 1990 |

**Câu 2.3 (Áp dụng).** Tính tổng tiền lớn nhất mà chương trình có thể in ra với miền nhập hợp lệ và cho biết phép cộng có thể gây tràn có dấu 32 bit hay không. Nếu miền nhập mở rộng đến N kWh thì N lớn nhất là bao nhiêu để tổng tiền vẫn không tràn? Nêu cách tính.

*Trả lời: tổng tiền lớn nhất in ra trong miền hợp lệ là khi x = 9999 --> Tong tien: 30089960 dong*
*Số có dấu 32 bit lớn nhất là: 2^31 - 1 --> ko tràn*
*Tổng tiền N = 909000 + (N - 400) * 3040 <= 2^31 - 1*
*N <= 2147790647 / 3040 = 706510 kWh*

**Câu 2.4 (Đánh giá).** So sánh thiết kế duyệt mảng của Sinh viên với cách viết sáu nhánh `if` riêng cho từng bậc: nêu ít nhất hai tiêu chí (ví dụ kích thước mã, khi biểu giá đổi, khi thêm bậc) và kết luận phương án nào hợp lý hơn cho thiết bị nhúng trong tình huống này.

*Trả lời:*
*Phương thức duyệt mảng:*
*- kích thước mã: cố định và nhỏ vì chỉ cần xử lí ở trong 1 vòng lặp, số liệu nằm ở vùng .data*
*- khi thêm bậc: linh hoạt vì chỉ cần thêm bớt bật cho các phần tử trong mảng ở .data*
*Phương thức viết 6 nhánh if:*
*- kích thước mã: nhiều vì phải viết nhiều lệnh + - * / cho từng if*
*- khi thêm bật: bắt buộc phải thêm các nhánh if nên sẽ phức tạp*

*--> phương án duyệt mảng hợp lí hơn*


Mars Messages
Run I/O
Clear
-- program is finished running --
Nhap so kWh (0..9999): 7
Tien dien: 11480 dong

# THÍ NGHIỆM 3 — Phát giai điệu bằng syscall MIDI

Thời lượng gợi ý: 25 phút · File nộp: Lab01_<MSSV>_nhac.asm · Điểm: 2,0

### Tình huống

Thiết bị cần phát một giai điệu ngắn để báo hoàn tất. Giai điệu lưu dưới dạng hai mảng (cao độ và độ dài) trong bộ nhớ dữ liệu, chương trình duyệt mảng và gọi syscall MIDI cho từng nốt. Giai điệu là chủ đề *Ode to Joy* của L. van Beethoven (thuộc phạm vi công cộng); cao độ theo số hiệu nốt MIDI (60 là C4).

**Bảng 13.** Dữ liệu giai điệu (độ dài tính theo đơn vị nhịp, 1 đơn vị = giá trị `150 + 10*b` ms ở Bảng 1)

| Nốt | Tên | Cao độ | Độ dài | Nốt | Tên | Cao độ | Độ dài |
|---|---|---|---|---|---|---|---|
| 1 | E4 | 64 | 2 | 9 | C4 | 60 | 2 |
| 2 | E4 | 64 | 2 | 10 | C4 | 60 | 2 |
| 3 | F4 | 65 | 2 | 11 | D4 | 62 | 2 |
| 4 | G4 | 67 | 2 | 12 | E4 | 64 | 2 |
| 5 | G4 | 67 | 2 | 13 | E4 | 64 | 3 |
| 6 | F4 | 65 | 2 | 14 | D4 | 62 | 1 |
| 7 | E4 | 64 | 2 | 15 | D4 | 62 | 4 |
| 8 | D4 | 62 | 2 | | | | |

### Đặc tả chương trình

- Hai mảng `.word` trong `.data` chứa cao độ và độ dài (đơn vị nhịp) của 15 nốt; số nốt cũng là một dữ liệu trong `.data`.
- Với mỗi nốt: cao độ thực phát bằng cao độ trong mảng cộng `a`; thời lượng (ms) bằng độ dài nhân với `150 + 10*b`; nhạc cụ và âm lượng 100 theo tham số cá nhân. In ra một dòng `<cao do> <ms>` rồi phát nốt bằng syscall 33.
- Đo thời gian chạy của cả vòng phát bằng syscall 30 (hiệu của hai lần đọc `$a0`). Cuối chương trình in `Tong du kien (ms): <so>` (chương trình tự cộng dồn thời lượng các nốt) và `Thoi gian do duoc (ms): <so>`.

**Bước 3.1 – Dự đoán.** Dùng dữ liệu của Sinh viên trước khi viết mã.

**DỰ ĐOÁN** Tổng thời lượng dự kiến của 15 nốt là bao nhiêu ms? Nếu thay syscall 33 bằng syscall 31 (giữ nguyên tham số), điều gì xảy ra với âm thanh và với thời gian đo được?

| Tổng dự kiến (ms) | Dự đoán khi thay 33 bằng 31 |
|---|---|
| 1 đơn vị nhịp : 150 + 10*7 = 220 ms<br>Tổng đơn vị nhịp = 2 * 12 + 3 + 1 + 4 = 32<br>Tổng thời lượng dụ kiện 15 nốt là: 220 * 32 = 7040 ms | - 33: phát đúng giai điệu, nốt này phát xong nốt tiếp theo mới phát, thời gian đo được khoảng 7040ms<br>- 31: phát giai điệu hỗn loạn, chồng lên nhau, thời gian đo được rất ngắn |

**Bước 3.2 – Lập trình và quan sát.** Viết `Lab01_<MSSV>_nhac.asm` theo đặc tả, chạy và ghi số liệu vào Bảng 14. Nếu máy không phát được âm thanh, vẫn làm đủ các bước: ghi "Không nghe được" ở cột Nghe thấy và dựa vào các dòng in ra và thời gian đo được.

**QUAN SÁT** Nghe giai điệu có đúng nhịp không? Đối chiếu dòng in cuối với dự đoán ở Bước 3.1.

**Bước 3.3 – So sánh ba cách phát.** Thử lần lượt ba cấu hình và điền Bảng 14: (A) syscall 33; (B) syscall 31 thay cho 33; (C) syscall 31 rồi ngay sau đó syscall 32 với `$a0` bằng độ dài nốt (ms).

**Bảng 14.** Ba cấu hình phát nốt

| Cấu hình | Nghe thấy | Thời gian đo được (ms) | Số dòng in ra | Nhận xét ngắn |
|---|---|---|---|---|
| A: 33 | Âm thanh có nhịp điệu | 7867 | 17 | Âm thanh có nhịp điệu. Dễ nghe. Có nốt cao nốt thấp, nghe được 15 nốt |
| B: 31 | Âm thanh 1 lần gần như phát toàn bộ 15 nốt trong khoảng thời gian ngắn | 2 | 17 | Âm thanh nghe chói tai, gần như nghe 1 lần duy nhất rất ngắn |
| C: 31 + 32 | Giống syscall 33 | 7059 | 17 | Giống syscall 33 |

**Câu 3.1 (Phân tích).** Giải thích chênh lệch (nếu có) giữa tổng dự kiến và thời gian đo được ở cấu hình A. Liệt kê các nguồn gây chênh lệch mà Sinh viên nhận ra.

*Trả lời: có chênh lệch vì:*
*- thời gian dự kiến chỉ tính tổng thời gian phát của các nốt nhạc còn thời gian đo được là bao gồm cả thời gian thực thi chương trình như: độ trễ thực thi lệnh mips, độ trễ xuất ra màn hình, đồ trễ khi tính toán,...*

**Câu 3.2 (Áp dụng).** Nếu mảng cao độ khai báo bằng `.byte` thay vì `.word`, lệnh nạp phần tử và cách tính địa chỉ phần tử thứ i thay đổi thế nào? Chọn `lb` hay `lbu` cho trường hợp cao độ có thể lên đến 127 và cho trường hợp tổng quát 0–255? Vì sao?

*Trả lời:*
*- dùng .byte thay.word thì offset = i luôn thay vì i * 4 vì 1 word = 4 bytes, lệnh nạp là lb hoặc lbu thay vì lw*
*- vì 1 byte = 8 bit nên lb sẽ đọc từ -128 tới 127, còn lbu đọc từ 0 tới 255*
*Nên cao độ max là 127 thì lb với lbu đều dc, còn tổng quát thì lbu vì số ko dấu lớn nhất là 2^8 - 1 = 255*

**Câu 3.3 (Đánh giá).** Dựa trên Bảng 14, thiết bị vừa phải phát nhạc vừa phải đọc cảm biến thì nên dùng cấu hình nào? Nêu một rủi ro khi dùng cấu hình B mà không chờ.

*Trả lời:*
*- nên dùng cấu hình B vì nó là phát nhạc bất đồng bộ, trả quyền lại CPU ngay lập tức để đọc cảm biến*
*- rủi ro dùng cấu hình B mà không chờ là: B phát cùng lúc tất cả các nốt nên khiến cho việc phát nhạc không được chính xác, âm thanh hỗn loạn, chói tai*

## THÍ NGHIỆM 4 — Đo số lệnh và giải mã lệnh trên chương trình của mình

Thời lượng gợi ý: 30 phút · Dùng chương trình `Lab01_<MSSV>_dien.asm` đã hoàn thành · Điểm: 2,0

**Tình huống**

Trước khi triển khai lên vi xử lý nhúng có tài nguyên hạn chế, nhóm phát triển cần ước lượng chi phí của chương trình theo dữ liệu vào và hiểu rõ mã máy sinh ra. Sinh viên đo số lệnh thực thi của chương trình tính tiền điện, xây dựng mô hình chi phí đơn giản và giải mã một số lệnh của chính chương trình mình.

**Bước 4.1 – Đo số lệnh thực thi.** Mở Instruction Counter (Connect to MIPS). Với mỗi x ở Bảng 15, đặt lại bộ đếm, chạy, ghi tổng số lệnh thực thi. Đo x = 10 và x = 120 trước.

**DỰ ĐOÁN** Từ hai số đo đầu tiên, đề xuất công thức N(x) và điền cột Dự đoán cho các hàng còn lại; sau đó đo các hàng còn lại và so sánh.

**Bảng 15.** Số lệnh thực thi theo số kWh nhập

| x (kWh) | Số bậc đã dùng hết | N dự đoán | N đo được | Lệch |
|---|---|---|---|---|
| 10 | 0 | — | 47 | — |
| 120 | 2 | — | 83 | — |
| 50 | 1 | 65 | 47 | 18 |
| 250 | 3 | 101 | 101 | 0 |
| 401 | 5 | 137 | 137 | 0 |

**Câu 4.1 (Phân tích).** Viết công thức N(x) của chương trình Sinh viên và giải thích ý nghĩa từng hệ số bằng cách đếm số lệnh trong phần chuẩn bị, trong thân vòng lặp mỗi bậc và trong phần in kết quả. Vì sao N không đổi với mọi x thuộc cùng một bậc?

*Trả lời:công thức N = 18k + 47 với k = số bật đã dùng hết*
*18 = 12 + 5 (12 + 5 là số lệnh thực thi trong vòng lặp bắt buộc xảy ra) +1 (là 1 trong 2 lệnh nhánh sẽ xảy ra)*
*47 = 16 + 13 (là số lệnh thực thi của xuất nhập) + 5 + 12 (số lệnh thực thi trong vòng lặp bắt buộc xảy ra)*
*Công thức đúng khi 0 <= x <= 9999*
*- N không đổi với mọi x thuộc cùng một bậc vì công thức xử dụng k là số bật đã dùng hết của x*

**Câu 4.2 (Đánh giá).** Nếu biểu giá có 60 bậc và x nằm ở bậc cuối, N xấp xỉ bao nhiêu? Đề xuất **một** thay đổi nhằm giảm số lệnh mỗi vòng, thực hiện và đo lại ở x = 401; báo cáo N trước, N sau và phần trăm giảm. Nếu không giảm được, lập luận vì sao thân vòng đã tối thiểu.

*Trả lời:*
*N sấp sĩ = 1127 lệnh*
*Đề xuất: gán $t4 = -1 ra ngoài vòng lặp*
*N trước = 137*
*N sau = 132*
*%giảm = 5 / 137 = 3.65%*

**Bước 4.2 – Giải mã lệnh của chương trình mình.** Trong Text Segment của chương trình tiền điện, chọn ba lệnh: một lệnh nhánh trong vòng lặp, một lệnh nạp từ mảng và một lệnh so sánh. Với mỗi lệnh, ghi địa chỉ, mã hex, tách các trường và **tự đọc ra lệnh trước khi nhìn cột Basic**.

QUAN SÁT So khớp kết quả giải mã với cột Basic. Nếu khác, tìm trường nào bị đọc sai.

**Bảng 16.** Giải mã ba lệnh của chương trình tiền điện

| Địa chỉ | Mã hex | Dạng lệnh | Các trường (tên = giá trị thập phân) | Lệnh đọc ra |
|---|---|---|---|---|
| 0x00400068 | 0x112c000c | I | Opcode = 4<br>Rs = 9<br>Rt = 12<br>Imm = 12 | beq $t1, $t4, end_loop |
| 0x0040064 | 0x8d890000 | I | Opcode = 23_hex = 35<br>Rs = 12<br>Rt = 9<br>Imm = 0 | lw $t1, 0($t4) |
| 0x0040001c | 0x2e0b2710 | I | Opcode = 11<br>Rs = 16<br>Rt = 11<br>Imm = 10000 | sltiu $t3, $s0, 10000 |

**Câu 4.3 (Áp dụng).** Với lệnh nhánh đã chọn, tính (địa chỉ đích − (địa chỉ lệnh + 4)) / 4 và so với trường immediate trong mã hex. Với trường immediate 16 bit, một lệnh nhánh đến được các địa chỉ trong khoảng bao nhiêu byte so với lệnh kế tiếp?

*Trả lời:*
*0x0040009c - (0x00400068 + 4) / 4 = 12 trùng với immediate trong mã hex*
*Immediate 16 bit --> biểu diễn trong khoảng [-2^15 ; 2^15 - 1] = [-32768 ; 32767]*
*Lệnh mips dài 1 word = 4 bytes*
*Địa chỉ câu lệnh luôn +thêm 4 tương đương mỗi địa chỉ câu lệnh kế tiếp tăng 4 bytes = imm * 4*
*--> lệnh nhánh đến các địa chỉ trong khoảng [-131072 ; 131068] bytes*

**Bảng 17.** Giả lệnh trong chương trình tiền điện

| Giả lệnh trong mã nguồn | Lệnh cơ bản sinh ra | Số lệnh |
|---|---|---|
| la $a0, nhap | lui $1, 0x00001001<br>ori $4, $1, 0x00000034 | 2 |
| j xu_ly | j 0x0040003c | 1 |
| ble $s0, $t1, skip_kWh | slt $1, $9, $16<br>beq $1, $0, 0x00000001 | 2 |

**Câu 4.4 (Đánh giá).** Chương trình của Sinh viên có bao nhiêu dòng lệnh nguồn và bao nhiêu lệnh cơ bản trong bộ nhớ? Nếu một giả lệnh nạp địa chỉ nằm trong thân vòng lặp thì nó ảnh hưởng thế nào đến N(x)? Nêu cách bố trí chương trình để tránh chi phí này.

*Trả lời:*
*- dòng lệnh nguồn: 45*
*- đòng lệnh cơ bản 52*
*- nạp địa chỉ (la) trong thân vòng lặp thì tăng thêm 2 lệnh cơ bản trong mỗi vòng lặp --> N(x) tăng thêm 2*m số vòng lặp*
*- chuyển các lệnh la ra ngoài vòng lặp và tính offset = cách cộng thêm 4 (số bytes lệch) thay vì nạp lại giá trị gốc hoặc tính offset nhiều lần*

# TỔNG KẾT VÀ NỘP BÀI (20 phút)

## Danh sách kiểm tra trước khi nộp

| ☑ | Nội dung |
|---|---|
| ☑ | File `Lab01_<MSSV>_dien.asm` Assemble không lỗi, có `.data`, `.text`, nhãn `main`, kết thúc bằng syscall 10 và chú thích vai trò thanh ghi. |
| ☑ | File `Lab01_<MSSV>_nhac.asm` Assemble không lỗi và in đúng khuôn các dòng yêu cầu. |
| ☑ | File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp vào các khung nét đứt. |
| ☑ | Số liệu trong báo cáo khớp với chương trình nộp (đơn giá, tham số cá nhân và kết quả đo). |
| ☑ | Đặt tên file báo cáo: `Lab01_<MSSV>_BaoCao.docx`; nộp ba file theo hướng dẫn của giảng viên. |

## Tự chấm

| Phần | Điểm tối đa | Tự chấm |
|---|---|---|
| TN 1 | 1,5 | 1,5 |
| TN 2 | 4,0 | 3,5 |
| TN 3 | 2,0 | 1,75 |
| TN 4 | 2,0 | 1,75 |
| Kết | 0,5 | 0,5 |

## Phản tư

**Câu R1 (Đánh giá).** Từ TN 1 và TN 4, Sinh viên rút ra điều gì về sự khác nhau giữa số dòng nguồn, số lệnh trong bộ nhớ và số lệnh thực thi? Nêu một tình huống cần quan tâm đến từng con số.

*Trả lời:*
*Số dòng nguồn: số lệnh người dùng lập trình trong Assembly (ko tính nhãn, commet, dòng trống)*
*Số lệnh trong bộ nhớ: tổng số lệnh cơ bản thực tế trong Text segment mà chương trình thực hiện*
*Số lệnh thực thi: số lệnh thực sự thực thi mà đi qua CPU, số phụ thuộc vào vòng lặp và các câu lệnh nhánh*

*Tình huống cần quan tâm:*
*Số dòng nguồn: đánh giá độ phức tạp của thuật toán*
*Số lệnh trong bộ nhớ: triển khai trên các thiết bị nhúng có bộ nhớ hạn chế*
*Số lệnh thực thi: tối ưu hóa tốc độ xử lí và mức tiêu thụ năng lượng*

**Câu R2 (Phân tích).** Từ TN 2, kiểu so sánh có dấu hay không dấu quyết định điều gì trong chương trình? Nêu một tình huống khác ngoài kiểm tra miền nhập mà chọn sai sẽ gây lỗi.

*Trả lời: so sánh không giấu đôi lúc sẽ bao quát luôn so sánh có dấu, từ đó giảm số lệnh cần để so sánh*
*So sánh có dấu: xét MSB bit 1 là âm, bit 0 là dương, so sánh không dấu: luôn coi là số dương*
*Tình huống: Địa chỉ bộ nhớ trong kiến trúc máy tính luôn là số dương, nếu so sánh có dấu (slt) thì nó sẽ nhầm tưởng là số âm nếu MSB là bit 1 từ đó chuong trình sai*

**Câu R3 (Đánh giá).** Nêu một lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

*Trả lời: lỗi: khi đo lệnh thực thi quên không bấm nút Reset trong cửa sổ công cụ Instruction Counter, từ đó số lệnh thực thi bị cộng dồn nên kết quả dự đoán sai (trong TN4)*
*Cách phát hiện: so sánh công thức làm ra và quan sát được trên Instruction Counter, đối chiếu và thử lại bằng cách bấm nút reset thì kết quả đúng là 101*
*Bài học rút ra: cần phải reset các công cụ thử nghiệm trước mỗi lần chạy khi có dữ liệu mới*

## Tài liệu tham khảo

- Slide bài giảng Chương 2 – Kiến trúc bộ lệnh (Khoa Kỹ thuật Máy tính, UIT).
- MARS 4.5, Help ▸ Help: các mục Instructions, Pseudo-instructions, Syscalls.